



BIT ENERGIAS RENOVÁVEIS E TECNOLOGIA

MARIA JÚLIA MELO SILVA

MEMORIAL TÉCNICO DE DESENVOLVIMENTO -

PORTAL DE SOLICITAÇÕES INTERNAS

**CAMPINA GRANDE - PB
2026**

SUMÁRIO

1. INTRODUÇÃO.....	4
1.1. Objetivo do Sistema.....	4
1.2. Stack Tecnológica.....	5
2. ARQUITETURA DA APLICAÇÃO.....	6
2.1. Arquitetura Backend.....	7
2.1.1. Estrutura de Pastas.....	7
2.1.2. Separação de Camadas.....	8
2.1.3. Server (Ponto de Entrada).....	9
2.1.4. Router (Rotas).....	9
2.1.5. Middlewares.....	10
2.1.6. Controllers.....	10
2.1.7. Validators.....	11
2.1.8. Config.....	11
2.1.9. Prisma e Database.....	12
2.1.10. Fluxo de Requisição.....	12
2.1.11. Padronização de Respostas e Erros.....	13
2.2. Arquitetura Front End.....	14
2.2.1. Estruturas de Pastas.....	14
2.2.2. Organização de páginas.....	15
2.2.3. Gerenciamento de Estado.....	16
2.2.4. Comunicação com a API.....	17
2.2.5. Padrões de Projeto Utilizados.....	18
2.3. Diagrama de Arquitetura.....	18
3. JUSTIFICATIVAS TÉCNICAS, CONCEITUAIS E ANÁLISE CRÍTICA.....	19
3.1. Tecnologias Utilizadas.....	19
3.2. Justificativa Técnica.....	21
3.2.1. Node.js e Express.....	21
3.2.2. Prisma ORM.....	21
3.2.3. PostgreSQL.....	22
3.2.4. JSON Web Token e Bcrypt.....	22
3.2.5. React e TypeScript.....	22
3.2.6. Vite e React Router.....	23
3.2.7. CSS.....	23
3.2.8. Bibliotecas e Ferramentas de Apoio.....	23
3.3. Fundamentação da Arquitetura.....	24
3.3.1. Arquitetura Geral do Sistema.....	24
3.3.2. Estrutura das Camadas.....	24
3.3.3. Mecanismo de Autenticação.....	25
3.3.4. Integração entre Frontend e Backend.....	25
3.3.5. Organização do código-fonte.....	25
3.4. Considerações Técnicas.....	26
3.4.1. Limitações Identificadas.....	26

3.4.2. Possíveis Aprimoramentos.....	26
3.4.3. Aprimoramento dos Requisitos.....	26
3.4.4. Considerações para Ambiente de Produção.....	27
4. MODELAGEM DO BANCO DE DADOS.....	27
4.1. Requisitos Funcionais.....	27
4.1.1. Autenticação e Controle de Acesso.....	27
4.1.2. Gestão de Solicitações.....	28
4.1.3. Consulta e Indicadores.....	29
4.2. Regras de Negócio (RN).....	29
4.2.1. Controle de Acesso e Visibilidade.....	29
4.2.2. Ciclo de Vida das Solicitações.....	29
4.2.3. Integridade e Validação.....	30
4.3. Modelo Lógico.....	30
4.4. Dicionário de Dados.....	32
4.4.1. Tabela Usuário.....	33
4.4.2. Tabela Categoria.....	33
4.4.3. Tabela solicitação.....	34
4.4.4. Tabela historico_status.....	35
4.4.5. Índices.....	36
5. EXECUÇÃO E DEPLOY.....	36
5.1. Visão Geral.....	36
5.2. Pré-requisitos.....	37
5.3. Ambiente Local (Desenvolvimento).....	37
5.3.1. Obtenção do código-fonte.....	37
5.3.2. Criação do Banco de Dados.....	37
5.3.3. Configuração do backend.....	38
5.3.4. Criação da estrutura do banco de dados.....	38
5.3.5. Configuração do frontend.....	39
5.3.6. Execução.....	39
5.4. Variáveis de Ambiente.....	39
5.5. Portas Utilizadas.....	40
5.6. Acesso e Credenciais de Demonstração.....	40
5.7. Problemas Comuns.....	41

1. INTRODUÇÃO

O Portal de Solicitações Internas é uma aplicação web full stack desenvolvida para centralizar o registro e o acompanhamento de demandas internas de uma organização. Por meio dele, colaboradores abrem solicitações direcionadas às áreas de TI, RH, Compras, Financeiro e Infraestrutura e acompanham cada uma delas desde a abertura até a conclusão, enquanto a equipe de atendimento gerencia o andamento dessas demandas em um único ambiente.

Este memorial registra o processo decisório adotado durante o desenvolvimento, as tecnologias escolhidas e suas justificativas, as decisões de arquitetura, a modelagem de dados, as regras de negócio implementadas e uma análise crítica sobre as limitações e possíveis evoluções da solução.

O sistema permite que usuários autenticados registrem solicitações informando título, descrição e categoria. A data de criação, o usuário solicitante e o status inicial ("Aberto") são preenchidos automaticamente pelo backend, sem intervenção do usuário. Cada solicitação percorre o fluxo:

Aberto → Em Atendimento → Concluído

Toda mudança de status é registrada em um histórico com autor, data e observação, garantindo a rastreabilidade das demandas.

A solução foi construída com arquitetura baseada em API REST, com separação entre frontend e backend, autenticação via token JWT e persistência em banco de dados relacional PostgreSQL. Essa organização favorece a manutenção, a clareza do código e a evolução independente de cada camada.

1.1. Objetivo do Sistema

O Portal de Solicitações Internas tem como objetivo organizar e padronizar o fluxo de demandas internas, oferecendo ao solicitante visibilidade sobre o andamento do seu pedido e à equipe de atendimento uma visão consolidada de todas as solicitações.

O sistema é destinado a dois perfis de usuário. O Colaborador abre solicitações e acompanha apenas as suas, podendo editá-las ou excluí-las enquanto estiverem com status "Aberto". O Atendente visualiza todas as solicitações, altera seus status e gerencia os usuários pela API (função exercida apenas por endpoints no backend).

O escopo do sistema abrange a gestão completa do ciclo de vida de uma solicitação que é: criação, edição, exclusão, consulta de detalhes, alteração de status, histórico, filtros e indicadores no dashboard.

1.2. Stack Tecnológica

O sistema de Solicitações Internas foi desenvolvido utilizando uma arquitetura moderna baseada em separação entre frontend e backend, com comunicação via API REST. A stack tecnológica foi escolhida visando desempenho, escalabilidade, organização de código e facilidade de manutenção.

No backend, a aplicação foi construída utilizando Node.js, que permite a execução de JavaScript no servidor, proporcionando alta performance em operações assíncronas e excelente integração com APIs. Como framework para estruturação das rotas e controle das requisições HTTP, foi utilizado o Express.js, responsável por organizar os endpoints do sistema, controlar middlewares e estruturar a lógica das requisições e respostas.

Para a camada de acesso ao banco de dados, o sistema utiliza o Prisma ORM, que atua como intermediador entre a aplicação e o banco relacional. O Prisma permite a definição tipada do modelo de dados, geração automática do cliente, controle de migrações e consultas mais seguras e organizadas. Isso reduz erros manuais de SQL e melhora a legibilidade do código.

O banco de dados utilizado é o PostgreSQL, um SGBD relacional robusto e de código aberto. A estrutura do banco é criada por scripts SQL versionados no repositório, atendendo à exigência de fornecer mecanismos explícitos de criação da estrutura necessária para a execução do sistema.

No frontend, a aplicação foi desenvolvida utilizando React, uma biblioteca JavaScript para construção de interfaces modernas e reativas. O React permite atualização dinâmica da interface sem recarregamento da página, proporcionando melhor experiência ao usuário. O projeto é gerenciado com Vite, que oferece ambiente de desenvolvimento rápido, build otimizado e melhor desempenho durante a execução local.

A autenticação do sistema é baseada em JWT (JSON Web Token). Após o login, o backend gera um token contendo informações como ID do usuário e perfil (role). Esse token é enviado ao front end e utilizado para autorizar requisições protegidas, controlar permissões e validar ações como edição e exclusão de reservas.

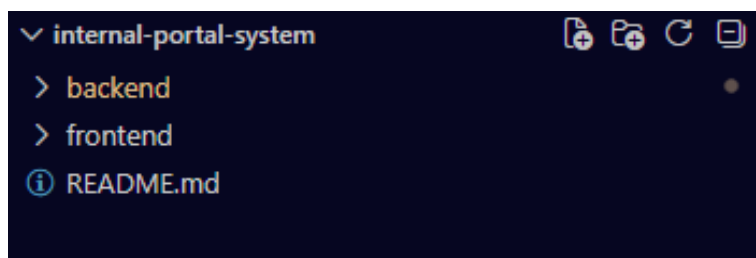
2. ARQUITETURA DA APLICAÇÃO

O Portal de Solicitações Internas foi desenvolvido seguindo uma arquitetura em camadas, com separação clara de responsabilidades entre roteamento, controle de acesso, regras de negócio, validação e acesso a dados. Essa organização facilita a manutenção, a leitura do código e a evolução de cada parte sem impacto nas demais.

A aplicação é dividida em duas grandes partes, mantidas em pastas independentes no mesmo repositório:

- **Backend (API REST)**
- **Frontend (Interface Web)**

Figura 1: Ilustração das pastas presentes no Sistema.



Fonte: Elaboração própria (2026).

A escolha por separar frontend e backend, em vez de uma aplicação monolítica com páginas renderizadas no servidor, se deve a três fatores. Primeiro, foi exigido uma API para acesso aos dados, e a separação torna essa API um contrato explícito, que poderia ser consumido por outros clientes no futuro (um aplicativo móvel, por exemplo). Segundo, cada parte pode ser executada, testada e publicada de forma independente. Terceiro, a separação reforça a divisão de responsabilidades: o backend concentra as regras de negócio e a segurança, enquanto o frontend se ocupa apenas da interação com o usuário.

A opção por manter as duas partes no mesmo repositório foi tomada para facilitar a avaliação e a execução: com um único git clone, terá acesso ao código completo, aos scripts do banco e à documentação.

Nesta seção, detalharemos a arquitetura do backend.

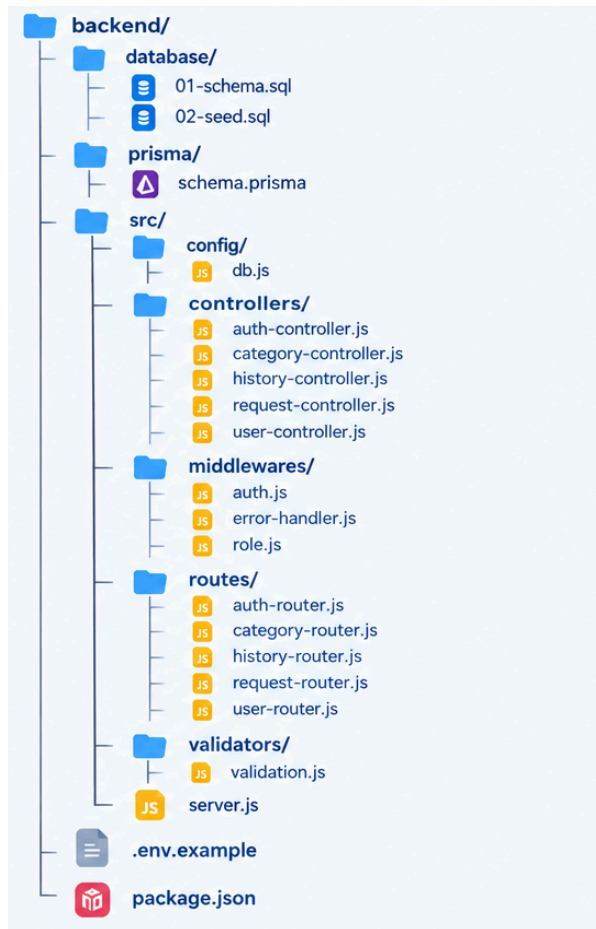
2.1. Arquitetura Backend

O backend foi desenvolvido utilizando Node.js com Express, estruturado em camadas organizadas por responsabilidade.

2.1.1. Estrutura de Pastas

A estrutura segue um padrão modular, em que cada domínio da aplicação (autenticação, usuários, categorias, solicitações e histórico) possui seu próprio arquivo de rota e de controller:

Figura 2: Ilustração modular do Backend.



Fonte: Elaboração Própria(2026).

A padronização de nomes (<domínio>-router.js e <domínio>-controller.js) permite localizar rapidamente o código de qualquer funcionalidade: para entender como funcionam as solicitações, basta abrir request-router.js e request-controller.js.

2.1.2. Separação de Camadas

A aplicação segue o princípio da separação de responsabilidades, em que cada camada tem uma função bem definida e se comunica apenas com a camada seguinte.

2.1.3. Server (Ponto de Entrada)

O arquivo **server.js** é o ponto de entrada da API. Ele é responsável por configurar o Express, habilitar a leitura de JSON, configurar o CORS, registrar as

rotas, aplicar os middlewares de erro e confirmar a conexão com o banco antes de iniciar o servidor.

As rotas são divididas em dois grupos: a rota pública de login (/api/auth), que não exige autenticação, e as rotas protegidas por JWT, que passam pelo middleware auth antes de chegar ao controller (/api/request, /api/category, /api/history e /api/user). A rota de usuários recebe ainda o middleware requirePerfil('ATENDENTE'), restringindo o acesso ao perfil de atendimento.

A API só começa a aceitar requisições após confirmar a conexão com o PostgreSQL. Caso a conexão falhe, o processo é encerrado com uma mensagem clara no terminal. Essa decisão evita que a aplicação suba em um estado inconsistente, em que todas as requisições falharam com erros genéricos.

2.1.4. Router (Rotas)

Responsáveis por definir os endpoints da API e associá-los aos respectivos controllers. A camada de rotas define os métodos HTTP (GET, POST, PUT, PATCH, DELETE) e aplica middlewares específicos de cada endpoint, como a restrição de perfil na alteração de status

As rotas não contêm nenhuma lógica de negócio, apenas o mapeamento entre endereço, método e função. Isso mantém os arquivos curtos e permite visualizar todos os endpoints de um domínio em poucas linhas.

A nomenclatura dos endpoints segue o padrão REST, usando o recurso como substantivo e o método HTTP como ação:

- **GET** /api/request ->listar.
- **POST** /api/request ->criar.
- **PUT** /api/request/:id -> editar.
- **DELETE** /api/request/:id -> excluir.

A alteração de status foi modelada como PATCH em um sub-recurso (/:id/status), por se tratar de uma atualização parcial com regras próprias, distinta da edição dos dados da solicitação.

2.1.5. Middlewares

A pasta `middlewares/` contém funções executadas antes ou depois dos controllers, responsáveis por preocupações transversais, ou seja, que se repetem em várias rotas:

- `auth.js` → valida o token JWT enviado no header `Authorization`, decodifica o payload (id, perfil e login) e o injeta em `req.user`. Se o token estiver ausente, inválido ou expirado, retorna HTTP 401.
- `role.js` → função `requirePerfil(...perfis)`, que recebe os perfis autorizados e retorna um middleware que bloqueia com HTTP 403 os usuários sem permissão.
- `error-handler.js` → contém dois middlewares finais: `notFound`, que responde HTTP 404 para rotas inexistentes, e `errorHandler`, que captura qualquer erro não tratado e o devolve no formato padronizado `{ message }`, sem expor detalhes internos ao cliente.

Concentrar autenticação, autorização e tratamento de erros em middlewares evita repetir essas verificações dentro de cada controller. Por exemplo, nenhum controller precisa verificar se o token existe, se a requisição chegou até ele, o usuário já está autenticado.

2.1.6. Controllers

A pasta `controllers/` contém a lógica principal da aplicação. Cada controller é responsável por um domínio específico:

- `auth-controller.js` → autenticação: valida as credenciais, compara a senha com o hash armazenado e gera o token JWT.
- `request-controller.js` → CRUD de solicitações, filtros da listagem, indicadores do dashboard e alteração de status com registro no histórico.
- `history-controller.js` → consulta do histórico de status de uma solicitação.
- `category-controller.js` → listagem das categorias ativas.
- `user-controller.js` → gerenciamento de usuários.

Os controllers recebem a requisição HTTP, validam dados, aplicam regras de negócio, chamam o Prisma para persistência e retornam a resposta ao cliente.

Em uma arquitetura mais extensa, seria comum criar uma camada de services separando as regras de negócio dos controllers. Para o tamanho deste projeto, com cinco domínios e regras concentradas principalmente nas solicitações, essa camada adicional aumentaria o número de arquivos sem ganho proporcional de clareza. Optou-se por manter as regras nos controllers, isolando em funções auxiliares o que se repete (validação de título, descrição e categoria, e tratamento dos erros do Prisma). Essa decisão é retomada na Análise Crítica como ponto de evolução.

2.1.7. Validators

A pasta validators/ contém funções reutilizáveis de validação e de controle de visibilidade:

- `parseId` → converte e valida identificadores recebidos na URL, aceitando apenas inteiros positivos.
- `parseDate` → valida datas no formato AAAA-MM-DD e as converte para início ou fim do dia, usado nos filtros por período.
- `visibilityWhere` → retorna o filtro de visibilidade conforme o perfil do Colaborador enxerga apenas as próprias solicitações, o Atendente enxerga todas.

A função `visibilityWhere` merece destaque por centralizar uma regra de segurança. Ela é aplicada na listagem, nos detalhes, no histórico e no dashboard. Assim, a regra "o colaborador só vê o que é seu" é escrita em um único lugar, e não corre o risco de ser esquecida em algum endpoint.

2.1.8. Config

A pasta config/ contém a configuração de infraestrutura:

- `db.js` → carrega as variáveis de ambiente do arquivo `.env`, monta a string de conexão com o PostgreSQL (aceitando tanto `DATABASE_URL` quanto

variáveis separadas de host, usuário, senha e banco) e exporta uma instância única do cliente Prisma, compartilhada por toda a aplicação.

2.1.9. Prisma e Database

O Prisma atua como ORM responsável pelo mapeamento objeto-relacional, execução de transações, validação de tipos e garantia de integridade referencial.

A camada de persistência é formada por duas pastas com papéis complementares:

- database/ → contém os scripts SQL que criam a estrutura do banco (01-schema.sql) e inserem os dados iniciais (02-seed.sql), como categorias e usuários de demonstração. Os scripts podem ser executados mais de uma vez sem duplicar dados.
- prisma/ → contém o schema.prisma, que mapeia as tabelas existentes para o cliente Prisma, permitindo consultas tipadas e o uso de relacionamentos no código, define as entidades, relacionamentos e configuração do banco de dados.

O Prisma oferece um mecanismo próprio de criação de tabelas (*migrations* ou db push). Ainda assim, optou-se por scripts SQL escritos manualmente como fonte oficial da estrutura do banco. Essa decisão atende diretamente ao requisito de fornecer scripts de criação das tabelas, torna a estrutura legível para qualquer pessoa que conheça SQL e permite definir restrições com precisão, como CHECK nos valores de status e perfil. O Prisma é utilizado apenas como camada de acesso aos dados, a partir do mapeamento dessas tabelas.

2.1.10. Fluxo de Requisição

O fluxo da aplicação segue o padrão:

Cliente (Frontend) → server.js → Middleware de autenticação → Router →
Middleware de perfil (quando aplicável) → Controller → Validators → Prisma
ORM → PostgreSQL

Fluxo detalhado, tomando como exemplo a alteração de status de uma solicitação:

- O Atendente envia PATCH /api/request/5/status com o novo status e uma observação
- O server.js direciona a requisição para o grupo /api/request, que exige autenticação
- O middleware auth.js valida o JWT e injeta os dados do usuário em req.user
- O router encaminha a requisição para changeStatus, passando antes pelo middleware requirePerfil('ATENDENTE')
- O controller executa:
 - Validação do código da solicitação, do status e do tamanho da observação
 - Regras de negócio: a solicitação existe, não está concluída e o novo status é a próxima etapa do fluxo
- O Prisma executa, em uma única transação, a atualização da solicitação e o registro no histórico
- O controller retorna a solicitação atualizada com HTTP 200
- Em caso de erro, a resposta segue sempre o formato { "message": "..."}, com o código HTTP correspondente (400, 401, 403, 404, 409 ou 500)

2.1.11. Padronização de Respostas e Erros

Todas as respostas de erro da API seguem o mesmo formato, { "message": "..."}, com mensagens em português pensadas para serem exibidas diretamente ao usuário. Os códigos HTTP foram escolhidos de acordo com a natureza do erro:

Quadro 1: Respostas de erros

Código	Situação
400	Dados inválidos ou regra de negócio violada (ex.: editar solicitação não aberta)
401	Token ausente, inválido ou expirado; usuário ou senha incorretos
403	Perfil sem permissão ou tentativa de alterar solicitação de outro usuário

404	Registro ou rota não encontrada
409	Conflito com o estado atual (ex.: alterar status de solicitação concluída)
500	Erro interno, sem exposição de detalhes técnicos

Fonte: Elaboração própria (2026).

Essa padronização simplifica o frontend, que trata todos os erros da mesma forma: lê o campo message e o exibe ao usuário. Os detalhes técnicos dos erros internos são registrados apenas no terminal do servidor, evitando expor informações sobre a estrutura do banco ou do código.

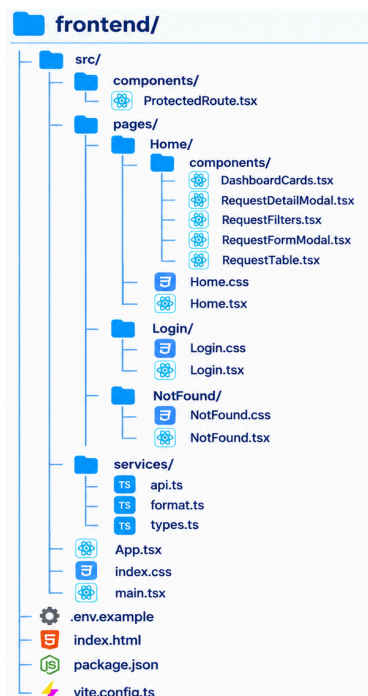
2.2. Arquitetura Front End

O frontend foi desenvolvido em React com TypeScript, utilizando Vite como ferramenta de build e React Router para a navegação. A comunicação com o backend é feita por uma função centralizada que utiliza o fetch nativo do navegador.

2.2.1. Estruturas de Pastas

A estrutura segue um padrão modular, organizado por páginas, em que componentes usados por uma única página ficam dentro da pasta desta página:

Figura 3: Ilustração modular do Front End.



A pasta `components/` na raiz guarda apenas o que é compartilhado entre páginas (hoje, a rota protegida). Componentes específicos, como os cards do dashboard e os modais de solicitação, ficam em `pages/Home/components/`, próximos de onde são usados. Essa organização por funcionalidade evita uma pasta global de componentes com arquivos sem relação entre si e deixa claro o escopo de cada um. Cada página também possui seu próprio arquivo CSS, mantendo os estilos próximos dos componentes que os utilizam.

2.2.2. Organização de páginas

Cada página é responsável por uma funcionalidade específica do sistema:

- `Login.tsx` → tela de autenticação, envia as credenciais para `/api/auth/login`, armazena o token e os dados do usuário e redireciona para a Home. Se o usuário já estiver autenticado, é redirecionado diretamente
- `Home.tsx` → página principal, reúne o dashboard, os filtros, a listagem de solicitações e os modais. É responsável por buscar os dados na API e coordenar os componentes filhos
- `NotFound.tsx` → página exibida para endereços inexistentes

A Home foi dividida em componentes menores, cada um com uma responsabilidade:

- `DashboardCards.tsx` → exibe os indicadores de total, abertas, em atendimento e concluídas
- `RequestFilters.tsx` → formulário de filtros por período, categoria, status e texto livre
- `RequestTable.tsx` → listagem com código, título, categoria, solicitante, data de abertura e status
- `RequestFormModal.tsx` → formulário utilizado para criar e editar solicitações
- `RequestDetailModal.tsx` → detalhes da solicitação, histórico de status e alteração de status (para o Atendente)

Optou-se por concentrar as funcionalidades em uma única página principal com modais, em vez de criar uma página para cada ação (listagem, cadastro,

edição, detalhes). Como todas as operações giram em torno da mesma listagem, os modais permitem que o usuário crie, edite ou consulte uma solicitação sem perder o contexto dos filtros aplicados. O mesmo componente de formulário atende criação e edição, variando apenas o modo, o que evita duplicação de código.

2.2.3. Gerenciamento de Estado

O gerenciamento de estado é feito localmente, utilizando os hooks nativos do React (`useState`, `useEffect` e `useCallback`). A página Home concentra o estado da tela (categorias, indicadores, solicitações, filtros e modal aberto) e o repassa aos componentes filhos por meio de propriedades (*props*). Os componentes filhos, por sua vez, comunicam ações de volta à Home por funções de callback. Não há biblioteca externa de estado global. Cada página é responsável por buscar, armazenar e atualizar seus próprios dados via chamadas à API.

O estado do modal foi modelado com um tipo TypeScript que define explicitamente os modos possíveis: criação, edição ou detalhes. Isso impede, em tempo de compilação, estados inválidos, como abrir o modo de edição sem uma solicitação associada.

2.2.4. Comunicação com a API

Todas as requisições passam pela função `apiFetch`, definida em `services/api.ts`. Essa função:

- Monta a URL completa a partir da variável de ambiente `VITE_API_URL`
- Recupera o token armazenado e o envia no header `Authorization: Bearer <token>`
- Define o cabeçalho `Content-Type: application/json`
- Converte a resposta para JSON e, em caso de erro, lança uma exceção `ApiError` contendo o código HTTP e a mensagem retornada pelo backend

Os tipos de entidades (usuário, categoria, solicitação, histórico e dashboard) ficam em `services/types.ts`, e as funções de formatação de datas e status ficam

em `services/format.ts`. Dessa forma, as páginas trabalham com dados tipados e não repetem lógica de formatação.

Quando a API responde com HTTP 401 (token expirado ou inválido), a Home remove os dados de sessão e redireciona o usuário para a tela de login, evitando que ele permaneça em uma tela que não consegue mais carregar dados.

Centralizar as requisições em uma única função garante que o token seja sempre enviado e que os erros sejam tratados da mesma forma em toda a aplicação. Uma alteração futura, como trocar a forma de armazenar o token ou adicionar um tempo limite às requisições, é feita em um único arquivo. O uso do fetch nativo, em vez de bibliotecas como Axios, evitou uma dependência adicional, já que a função `apiFetch` cobre as necessidades do projeto.

2.2.5. Padrões de Projeto Utilizados

Durante o desenvolvimento, alguns padrões de projeto foram aplicados de forma intencional:

- **Arquitetura em camadas:** separação entre rotas, middlewares, controllers, validadores e persistência, em que cada camada depende apenas da seguinte
- **Middleware (cadeia de responsabilidade):** autenticação, autorização por perfil e tratamento de erros são funções encadeadas pelo Express, em que cada uma decide se a requisição segue adiante ou é interrompida
- **Factory Function:** `requirePerfil(...perfis)` recebe os perfis autorizados e fabrica um middleware configurado, permitindo reutilizar a mesma lógica com perfis diferentes
- **Singleton:** uma única instância do cliente Prisma é criada em `config/db.js` e compartilhada por toda a aplicação, evitando abrir múltiplos conjuntos de conexões com o banco
- **Unit of Work (transação):** operações que alteram mais de uma tabela são executadas atomicamente, garantindo consistência entre solicitação e histórico

- **Wrapper:** a função `apiFetch` envolve o `fetch` nativo, adicionando autenticação e tratamento de erros padronizado
- **Composição de componentes:** a `Home` é composta por componentes menores e especializados, e o `ProtectedRoute` envolve páginas que exigem autenticação

2.3. Diagrama de Arquitetura

Figura 4: Diagrama de Arquitetura.



Fonte: Elaboração no Draw.io (2026).

Fluxo principal, a cadeia completa:

Usuário → Frontend → API Backend → Banco de Dados. O frontend nunca acessa o banco diretamente: toda leitura e escrita passa pela API, que aplica autenticação, permissões, validações e regras de negócio antes de qualquer operação.

3. JUSTIFICATIVAS TÉCNICAS, CONCEITUAIS E ANÁLISE CRÍTICA

Esta seção registra o processo decisório adotado no desenvolvimento do Portal de Solicitações Internas. São apresentadas as tecnologias utilizadas e suas justificativas técnicas, as decisões arquiteturais que orientaram a construção da solução e uma análise crítica acerca de suas limitações e possíveis evoluções.

3.1. Tecnologias Utilizadas

As tecnologias foram selecionadas considerando a natureza da aplicação, o prazo estabelecido para o desenvolvimento e a experiência prévia de outros

projetos já desenvolvidos. O Quadro 2 relaciona todas as tecnologias, bibliotecas e ferramentas empregadas no projeto.

Quadro 2 - Tecnologias utilizadas no projeto

Categoria	Tecnologia	Finalidade no projeto
Ambiente de execução	Node.js	Execução da API e das ferramentas de build
Gerenciador de pacotes	npm	Instalação de dependências e execução de scripts
Linguagem (backend)	JavaScript (módulos ES)	Implementação da API
Linguagem (frontend)	TypeScript	Tipagem estática da interface
Framework backend	Express	Rotas, middlewares e requisições HTTP
Mapeamento objeto-relacional	Prisma ORM	Acesso aos dados e execução dos scripts SQL
Banco de dados	PostgreSQL	Persistência dos dados
Extensão do banco de dados	pgcrypto	Geração de hash das senhas nos dados iniciais
Autenticação	jsonwebtoken (JWT)	Emissão e validação dos tokens de sessão
Criptografia	bcryptjs	Geração e verificação de hash de senhas
Segurança HTTP	CORS	Controle das origens autorizadas
Configuração	dotenv	Carregamento das variáveis de

		ambiente
Biblioteca frontend	React e React DOM	Construção da interface
Roteamento	React Router	Navegação e proteção de rotas
Ferramenta de build	Vite e @vitejs/plugin-react	Servidor de desenvolvimento e versão de produção
Estilização	CSS	Layout e responsividade
Administração do banco	pgAdmin	Criação do banco e execução manual dos scripts
Controle de versão	Git e GitHub	Versionamento e disponibilização da entrega

Fonte:Elaboração própria (2026).

3.2. Justificativa Técnica

As justificativas a seguir partem das necessidades concretas do portal, considerando, para cada tecnologia, o motivo da escolha, os benefícios para o cenário proposto, as vantagens em relação às alternativas e o impacto na manutenção, na escalabilidade ou na produtividade.

3.2.1. Node.js e Express

O Node.js foi adotado por permitir o uso de JavaScript no servidor, mesma família de linguagem do frontend, e pela experiência prévia da autora com a plataforma. Como o portal realiza predominantemente operações de leitura e escrita em banco de dados, o modelo assíncrono e não bloqueante do Node.js atende múltiplas requisições simultâneas com baixo consumo de recursos. Em comparação com Java (Spring Boot) e C# (.NET), demanda menos configuração e código estrutural para uma API deste porte.

O Express foi escolhido por ser o framework mais consolidado do ecossistema, e seu modelo de middlewares permitiu implementar autenticação, autorização e tratamento de erros como funções encadeadas e reutilizáveis. Alternativas como o NestJS impõem estrutura e curva de aprendizado desproporcionais a cinco domínios de negócio. O uso de uma mesma linguagem nas duas camadas reduz a alternância de contexto e favorece a produtividade em equipes reduzidas.

3.2.2. Prisma ORM

O Prisma foi adotado por oferecer consultas tipadas, legíveis e protegidas contra injeção de SQL. Recursos como transações e escrita aninhada foram aplicados diretamente nas regras de negócio, garantindo que a solicitação e seu histórico sejam gravados de forma atômica. Em relação ao Sequelize e ao TypeORM, possui sintaxe mais previsível e códigos de erro padronizados, aproveitados no tratamento de exceções. O arquivo schema.prisma atua ainda como referência do mapeamento, facilitando a manutenção.

3.2.3. PostgreSQL

O PostgreSQL foi escolhido em atendimento ao requisito de persistência em banco SQL, por ser um sistema relacional robusto, gratuito, de código aberto e de instalação simples no ambiente. Diferentemente do SQL Server e do Oracle, dispensa licenciamento e, em relação ao MySQL, oferece suporte mais completo a restrições de integridade e tipos de dados. Bancos não relacionais foram descartados por não atenderem ao requisito. Por ser amplamente utilizado em produção, o PostgreSQL comporta o crescimento do volume de dados sem necessidade de substituição da tecnologia.

3.2.4. JSON Web Token e Bcrypt

O padrão JWT foi adotado por permitir o controle de sessão sem armazenamento de estado no servidor. O token contém o identificador, o login e o perfil do usuário, possibilitando a aplicação das permissões sem consulta a uma tabela de sessões, e sua validade de oito horas corresponde a uma jornada de trabalho. Em comparação com sessões armazenadas no servidor, dispensa

estrutura adicional de persistência e favorece a escalabilidade horizontal, visto que múltiplas instâncias da API podem validar o mesmo token.

A biblioteca `bcryptjs` foi utilizada para o armazenamento das senhas exclusivamente na forma de hash. Algoritmos de propósito geral, como MD5 e SHA-256, foram descartados por serem vulneráveis a ataques de força bruta, e a versão em JavaScript puro foi preferida à biblioteca `bcrypt` original por dispensar compilação nativa, eliminando possíveis falhas de instalação.

3.2.5. React e TypeScript

O React foi adotado por seu modelo baseado em componentes, pela ampla adoção no mercado e pela experiência prévia. A página principal, que reúne indicadores, filtros, listagem e janelas modais, pôde ser dividida em componentes independentes e reutilizáveis, como o formulário único de criação e edição. O Angular mostra-se excessivamente robusto para uma interface deste porte, e a renderização no servidor exigiria o recarregamento da página a cada ação.

O TypeScript complementa essa escolha ao adicionar tipagem estática, permitindo identificar erros durante o desenvolvimento. Os tipos das entidades são definidos em um único arquivo, e o estado das janelas modais foi modelado de forma a impedir estados inválidos em tempo de compilação, o que confere maior segurança às refatorações.

3.2.6. Vite e React Router

O Vite foi adotado como ferramenta de build por ser a opção atualmente recomendada para projetos React, com inicialização rápida, atualização instantânea durante o desenvolvimento e suporte nativo a variáveis de ambiente. O Create React App foi descontinuado, e o Next.js agregaria recursos de renderização no servidor desnecessários a um sistema interno. O React Router, biblioteca de referência para roteamento em React, define as rotas da aplicação e viabiliza o componente de rota protegida, mantendo a navegação declarada em um único arquivo.

3.2.7. CSS

A estilização foi desenvolvida em CSS próprio, sem frameworks, em razão do número reduzido de telas e elementos visuais. A abordagem proporcionou controle integral sobre o layout e responsividade implementada por media queries, sem dependências adicionais. Bootstrap e Tailwind CSS aceleram interfaces extensas, porém acrescentam dependências e tendem a um padrão visual genérico. Ressalta-se que a ampliação da interface demandaria a criação de componentes visuais reutilizáveis ou a adoção de um framework.

3.2.8. Bibliotecas e Ferramentas de Apoio

As demais tecnologias desempenham funções de auxiliar as tecnologias utilizadas:

- **CORS** → Autoriza apenas as origens definidas em variável de ambiente, evitando a liberação irrestrita de acesso à API.
- **dotenv** → Separa configuração e código, mantendo credenciais fora do repositório e permitindo o uso do mesmo código em diferentes ambientes.
- **pgcrypto** → Permite que o script SQL crie usuários com senhas protegidas por bcrypt, sem armazenamento em texto plano, mantendo a instalação restrita a scripts SQL.
- **oxlint** → Identifica problemas recorrentes no código com tempo de execução inferior ao do ESLint e sem configuração extensa.
- **npm** → Padroniza a instalação de dependências e a execução dos scripts de banco, de desenvolvimento e de build.
- **pgAdmin** → Oferece alternativa gráfica para criação do banco e execução dos scripts, facilitando a instalação.
- **Git e GitHub** → Registram a evolução do projeto e reúnem código, *scripts* e documentação em um único repositório.

3.3. Fundamentação da Arquitetura

Esta subseção sintetiza as decisões arquiteturais adotadas, cujos detalhes de implementação encontram-se na seção 2.

3.3.1. Arquitetura Geral do Sistema

A aplicação foi estruturada em três componentes independentes: frontend, backend (API REST) e banco de dados. O frontend não acessa o banco diretamente, sendo toda operação intermediada pela API, que concentra as regras de negócio e a segurança. Essa separação torna a API um contrato explícito, permite executar e publicar cada componente de forma independente e delimita com clareza as responsabilidades de cada camada.

3.3.2. Estrutura das Camadas

O backend foi organizado nas camadas de rotas, middlewares, controllers, validadores e persistência, cada uma comunicando-se apenas com a subsequente. As rotas não possuem conhecimento das regras de negócio, e os controllers não verificam a autenticação, realizada previamente pelos middlewares. A ausência de uma camada de serviços (services) independente foi uma decisão deliberada, em função do porte do projeto, conforme exposto na seção 2.1.6

3.3.3. Mecanismo de Autenticação

A autenticação adota o modelo baseado em token sem estado. As credenciais são validadas pelo backend por meio da comparação com o hash armazenado e, em caso de sucesso, é emitido um JWT com validade de oito horas, enviado pelo frontend no cabeçalho Authorization das requisições seguintes. A mensagem de erro é idêntica para usuário inexistente e senha incorreta, de modo a não revelar a existência de determinado login.

A autorização ocorre em dois níveis: por perfil, restringindo ao Atendente a alteração de status e a gestão de usuários, e por propriedade, limitando ao Colaborador a visualização das próprias solicitações e reservando ao solicitante a edição e a exclusão. Todas as permissões são verificadas no backend, cabendo ao frontend apenas ocultar ações indisponíveis.

3.3.4. Integração entre Frontend e Backend

A comunicação segue o padrão REST, com dados em formato JSON e operações definidas pelos métodos HTTP. Os filtros da listagem são transmitidos como parâmetros de consulta, e as respostas de erro seguem formato único, composto por mensagem descritiva e código de status HTTP. No frontend, as requisições são centralizadas em uma única função, responsável por anexar o token, tratar erros e redirecionar o usuário em caso de sessão expirada. Os endereços da API e as origens autorizadas são definidos por variáveis de ambiente.

3.3.5. Organização do código-fonte

O backend adota organização por responsabilidade, com diretórios correspondentes às camadas e um arquivo por domínio, nomeados segundo padrão uniforme. O frontend adota organização por funcionalidade, na qual cada página possui diretório próprio com seus componentes e estilos, permanecendo em diretórios globais apenas os elementos compartilhados. O repositório contempla, ainda, os scripts SQL, os arquivos de exemplo das variáveis de ambiente e o README com as instruções de execução.

3.4. Considerações Técnicas

As escolhas discutidas a seguir foram tomadas de forma consciente, considerando o prazo e o escopo do projeto, e indicam os principais pontos de evolução da solução.

3.4.1. Limitações Identificadas

No aspecto de segurança, o token de autenticação é armazenado no localStorage, área acessível por JavaScript e, portanto, vulnerável a ataques de injeção de scripts. Por se tratar de um mecanismo sem estado, o token também não pode ser revogado antes de sua expiração, de modo que um usuário desativado mantém o acesso por até oito horas. A aplicação não dispõe, ainda, de renovação de sessão nem de limitação de tentativas de autenticação. Quanto

às funcionalidades, a listagem não possui paginação, a exclusão de solicitações é definitiva e a gestão de usuários está disponível apenas por meio da API.

3.4.2. Possíveis Aprimoramentos

Entre as evoluções previstas, destacam-se a paginação da listagem, a atribuição de solicitações a atendentes responsáveis, a definição de prioridades e prazos de atendimento e o envio de notificações por correio eletrônico a cada alteração de status. No âmbito técnico, recomenda-se a implementação de testes automatizados, a containerização da aplicação com Docker e a adoção de um pipeline de integração e entrega contínuas.

3.4.3. Aprimoramento dos Requisitos

Alguns requisitos do enunciado admitem mais de uma interpretação e exigiram decisões de projeto. Como não foi definido o responsável pela alteração de status, optou-se pela criação do perfil Atendente, impedindo que o próprio solicitante conclua sua demanda. Da mesma forma, a edição e a exclusão foram restritas ao autor da solicitação, e o fluxo de status foi definido como linear e sem retorno, não admitindo reabertura ou cancelamento. Em um contexto real, esses pontos seriam validados junto à área de negócio.

3.4.4. Considerações para Ambiente de Produção

Em um ambiente corporativo, a autenticação seria integrada ao serviço de diretório da organização, como o Active Directory, permitindo o uso das credenciais institucionais, e o token seria armazenado em cookie protegido, com possibilidade de revogação. As alterações no banco de dados seriam controladas por migrations versionadas, as regras de negócio seriam isoladas em uma camada de serviços coberta por testes e a publicação ocorreria por pipeline automatizado, com ambientes segregados de desenvolvimento, homologação e produção.

4. MODELAGEM DO BANCO DE DADOS

Esta seção apresenta a modelagem do banco de dados do Portal de Solicitações Internas, construída a partir dos requisitos funcionais e das regras de negócio estabelecidos no desafio. A modelagem resultou em quatro entidades: usuário, categoria, solicitação e histórico de status.

Como estratégia de modelagem, optou-se por garantir a integridade dos dados no próprio banco, e não apenas na aplicação, por meio de chaves estrangeiras, campos obrigatórios, restrições de unicidade e restrições CHECK. As categorias foram modeladas em tabela própria, uma vez que o enunciado as apresenta como sugestões, o que permite incluir ou desativar categorias sem alteração do código-fonte. O histórico de status foi modelado em tabela independente, registrando cada transição com seu autor, data e observação, de modo a atender ao acompanhamento da evolução das solicitações até a conclusão. A estrutura é criada pelo script 01-schema.sql, e os dados iniciais pelo script 02-seed.sql.

4.1. Requisitos Funcionais

4.1.1. Autenticação e Controle de Acesso

- **RF01 – Autenticação:** o sistema deve permitir o acesso por meio de usuário e senha, validando a senha informada com o hash armazenado no banco de dados.
- **RF02 – Controle de Sessão:** após a autenticação, deve ser emitido um token JWT contendo identificador, login e perfil do usuário, com validade de oito horas.
- **RF03 – Encerramento de Sessão:** o sistema deve permitir que o usuário encerre sua sessão, removendo as informações de autenticação armazenadas no navegador.
- **RF04 – Restrição de Acesso:** apenas usuários autenticados podem acessar as funcionalidades do sistema.
- **RF05 – Perfis de Acesso:** o sistema deve diferenciar os perfis Colaborador, responsável por registrar solicitações, e Atendente, responsável por conduzir seu atendimento.

- **RF06 – Gestão de Usuários:** o Atendente deve poder cadastrar, consultar, editar, desativar e excluir usuários por meio da API.

4.1.2. Gestão de Solicitações

- **RF07 – Cadastro de Solicitação:** o usuário deve poder registrar uma solicitação informando título, descrição e categoria.
- **RF08 – Preenchimento Automático:** no cadastro, o sistema deve preencher automaticamente a data de criação, o usuário solicitante e o status inicial "Aberto".
- **RF09 – Edição de Solicitação:** o sistema deve permitir a edição de título, descrição e categoria de solicitações com status "Aberto".
- **RF10 – Exclusão de Solicitação:** o sistema deve permitir a exclusão de solicitações com status "Aberto".
- **RF11 – Listagem de Solicitações:** o sistema deve exibir as solicitações contendo código, título, categoria, solicitante, data de abertura e status.
- **RF12 – Consulta de Detalhes:** o sistema deve permitir a visualização dos dados completos de uma solicitação, incluindo seu histórico de status.
- **RF13 – Alteração de Status:** o sistema deve permitir a alteração do status das solicitações, com registro de observação opcional.
- **RF14 – Histórico de Status:** toda alteração de status deve ser registrada com o status anterior, o novo status, o usuário responsável, a data e a observação.

4.1.3. Consulta e Indicadores

- **RF15 – Filtros de Pesquisa:** o sistema deve permitir a pesquisa de solicitações por período, categoria, status e texto livre no título, de forma combinada.
- **RF16 – Dashboard:** o sistema deve exibir a quantidade total de solicitações e as quantidades de solicitações abertas, em atendimento e concluídas.

4.2. Regras de Negócio (RN)

4.2.1. Controle de Acesso e Visibilidade

- **RN01 – Visibilidade por Perfil:** o Colaborador visualiza apenas as solicitações que registrou, enquanto o Atendente visualiza todas as solicitações do sistema. A regra aplica-se à listagem, aos detalhes, ao histórico e aos indicadores do dashboard.
- **RN02 – Propriedade da Solicitação:** somente o usuário que registrou a solicitação pode editá-la ou excluí-la.
- **RN03 – Exclusividade do Atendente:** a alteração de status e a gestão de usuários são permitidas exclusivamente ao perfil Atendente.
- **RN04 – Usuários Inativos:** usuários desativados não podem se autenticar no sistema.
- **RN05 – Proteção do Próprio Usuário:** o Atendente não pode excluir o próprio cadastro.

4.2.2. Ciclo de Vida das Solicitações

- **RN06 – Status Inicial:** toda solicitação é criada com o status "Aberto", acompanhada do registro inicial no histórico.
- **RN07 – Fluxo de Status:** as transições seguem obrigatoriamente a sequência Aberto, Em Atendimento e Concluído, não sendo permitido avançar diretamente de "Aberto" para "Concluído".
- **RN08 – Status Final:** solicitações concluídas não podem ter seu status alterado.
- **RN09 – Restrição de Edição e Exclusão:** solicitações com status "Em Atendimento" ou "Concluído" não podem ser editadas nem excluídas.
- **RN10 – Atomicidade do Histórico:** a alteração de status e seu registro no histórico são gravados na mesma transação, de modo que nunca exista alteração sem o respectivo registro.

4.2.3. Integridade e Validação

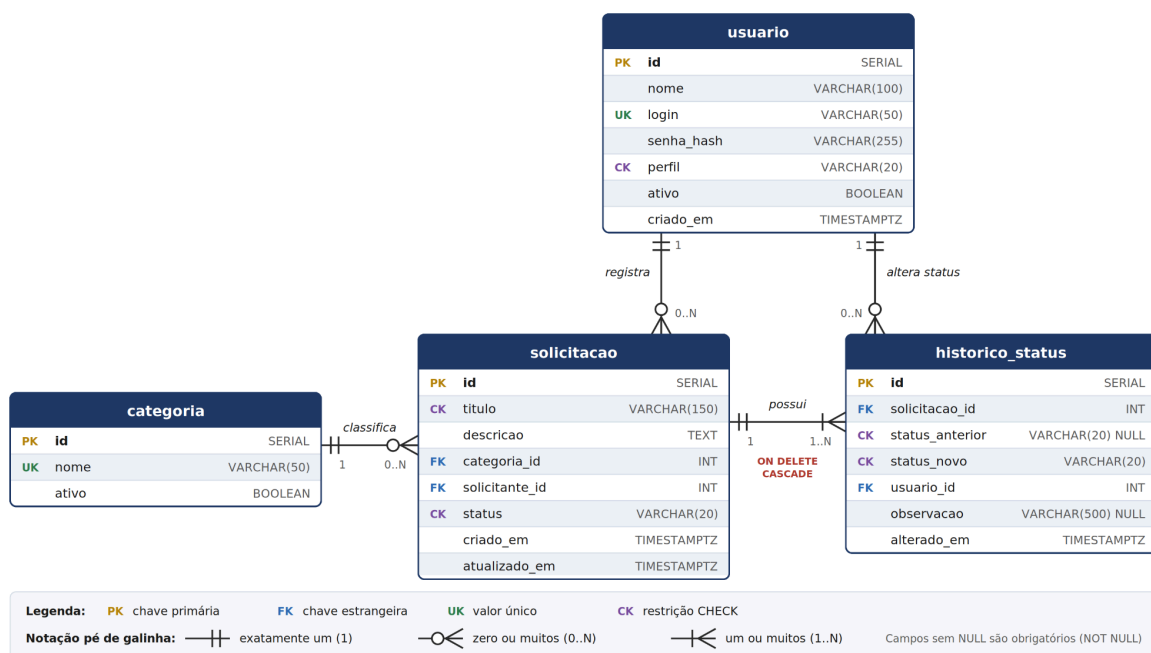
- **RN11 – Validação do Título:** o título deve conter entre 3 e 150 caracteres.
- **RN12 – Campos Obrigatórios:** título, descrição e categoria são obrigatórios no cadastro da solicitação.
- **RN13 – Categorias Ativas:** apenas categorias ativas podem ser associadas a novas solicitações ou a solicitações editadas.

- **RN14 – Unicidade do Login:** não podem existir dois usuários com o mesmo login.
- **RN15 – Observação de Status:** a observação registrada na alteração de status é opcional e limitada a 500 caracteres.
- **RN16 – Filtro por Período:** o período é aplicado sobre a data de abertura da solicitação, no formato AAAA-MM-DD, e a data inicial não pode ser posterior à data final.
- **RN17 – Exclusão em Cascata:** a exclusão de uma solicitação remove também os registros de histórico a ela vinculados.

4.3. Modelo Lógico

O modelo lógico do banco de dados, apresentado na Figura 5, representa as quatro tabelas do sistema com seus atributos, tipos de dados, chaves e restrições, bem como os relacionamentos estabelecidos entre elas. O diagrama foi elaborado a partir do script 01-schema.sql, de modo que reflete fielmente a estrutura implementada no PostgreSQL. Adotou-se a notação pé de galinha (crow's foot), na qual os símbolos posicionados nas extremidades de cada linha indicam a cardinalidade mínima e máxima do relacionamento.

Figura 5: Modelo lógico do banco de dados.



Fonte: Elaboração própria (2026).

A tabela solicitacao ocupa a posição central do modelo, por concentrar o objeto principal do sistema, e relaciona-se com as demais entidades da seguinte forma:

- **Categoria e Solicitação (1 : 0..N):** cada solicitação pertence a exatamente uma categoria, referenciada pela chave estrangeira categoria_id, enquanto uma categoria pode estar associada a nenhuma ou a várias solicitações. A cardinalidade mínima zero permite cadastrar novas categorias antes que existam demandas vinculadas a elas.

- **Usuário e Solicitação (1 : 0..N):** cada solicitação é registrada por exatamente um usuário, referenciado pela chave estrangeira solicitante_id. Um usuário pode não possuir solicitações, situação comum ao perfil Atendente, cuja função é conduzir o atendimento, e não registrar demandas.

- **Solicitação e Histórico de Status (1 : 1..N):** cada registro de histórico pertence a exatamente uma solicitação, referenciada pela chave estrangeira solicitacao_id. A cardinalidade mínima do lado do histórico é um, e não zero, em razão da regra de negócio RN06: toda solicitação é criada juntamente com seu registro inicial no histórico, na mesma transação. Ressalta-se que a chave estrangeira, isoladamente, admitiria uma solicitação sem histórico; a cardinalidade mínima é garantida pela camada de aplicação. O relacionamento utiliza exclusão em cascata (ON DELETE CASCADE), de forma que a remoção de uma solicitação elimina também seus registros de histórico, conforme a RN17.

- **Usuário e Histórico de Status (1 : 0..N):** cada alteração de status é atribuída a exatamente um usuário, referenciado pela chave estrangeira usuario_id, o que garante a rastreabilidade do autor de cada transição. Um usuário pode não ter realizado nenhuma alteração.

Além das chaves primárias e estrangeiras, o modelo explicita as restrições de unicidade (UK) nos campos login, da tabela usuario, e nome, da tabela categoria, que impedem registros duplicados, conforme a RN14. Também evidencia as restrições CHECK (CK), que limitam os valores aceitos nos campos perfil, status, status_anterior e status_novo aos domínios definidos nas regras de negócio, e o tamanho mínimo do campo titulo. Os campos não indicados como NULL são obrigatórios. Com isso, a integridade dos dados é assegurada no próprio banco de dados, independentemente da aplicação que o acesse.

O relacionamento entre usuário e solicitação foi modelado diretamente na tabela solicitacao, e não em uma tabela associativa, pois cada solicitação possui um único solicitante. Da mesma forma, o histórico foi mantido em tabela própria, em vez de colunas adicionais na solicitação, por se tratar de um relacionamento de um para muitos que precisa armazenar um número indeterminado de transições, cada uma com autor, data e observação.

4.4. Dicionário de Dados

O dicionário de dados descreve cada tabela do banco, com o nome, o tipo, o tamanho, a obrigatoriedade e a descrição de cada campo. Os tamanhos dos campos do tipo VARCHAR são expressos em número máximo de caracteres.

4.4.1. Tabela Usuário

Tabela que armazena os usuários com acesso ao sistema e seus respectivos perfis.

Quadro 3 - Dicionário de dados da tabela usuario

Nome	Tipo	Obrigatório	Descrição
id	SERIAL (INT)	✓	Chave primária, gerada automaticamente
nome	VARCHAR	✓	Nome completo do usuário

login	VARCHAR	✓	Identificador de acesso, de valor único
senha_hash	VARCHAR	✓	Hash bcrypt da senha do usuário
perfil	VARCHAR	✓	Perfil de acesso; aceita apenas "COLABORADOR" ou "ATENDENTE"
ativo	BOOLEAN	✓	Indica se o usuário pode acessar o sistema, padrão verdadeiro
criado_em	TIMESTAMPTZ	✓	Data e hora de cadastro, padrão data e hora atual

Fonte: Elaboração própria (2026).

4.4.2. Tabela Categoria

Tabela que armazena as categorias às quais as solicitações podem ser direcionadas.

Quadro 4 - Dicionário de dados da tabela categoria

Nome	Tipo	Obrigatório	Descrição
id	SERIAL (INT)	✓	Chave primária, gerada automaticamente
nome	VARCHAR	✓	Nome da categoria, de valor único
ativo	BOOLEAN	✓	Indica se a categoria pode ser associada a solicitações, padrão verdadeiro

Fonte: Elaboração própria (2026).

4.4.3. Tabela solicitação

Tabela que armazena as solicitações registradas pelos usuários.

Quadro 5 - Dicionário de dados da tabela solicitação

Nome	Tipo	Obrigatório	Descrição
id	SERIAL (INT)	✓	Chave primária, corresponde ao código da solicitação
titulo	VARCHAR	✓	Título da solicitação, com no mínimo 3 caracteres
descricao	TEXT	✓	Descrição detalhada da demanda
categoria_id	INT	✓	Chave estrangeira de categoria
solicitante_id	INT	✓	Chave estrangeira de usuário, usuário que registrou a solicitação
status	VARCHAR	✓	Situação atual, aceita "ABERTO", "EM_ATENDIMENTO" ou "CONCLUIDO"; padrão "ABERTO"
criado_em	TIMESTAMPTZ	✓	Data e hora de abertura, padrão data e hora atual
atualizado_em	TIMESTAMPTZ	✓	Data e hora da última alteração

Fonte: Elaboração própria (2026).

4.4.4. Tabela historico_status

Tabela que armazena o registro de cada alteração de status das solicitações.

Quadro 6 - Dicionário de dados da tabela historico_status

Nome	Tipo	Obrigatório	Descrição
id	SERIAL (INT)	✓	Chave primária,

			gerada automaticamente
solicitacao_id	INT	✓	Chave estrangeira de solicitação, com exclusão em cascata
status_anterior	VARCHAR	✗	Status antes da alteração, nulo no registro de criação
status_novo	VARCHAR	✓	Status após a alteração
usuario_id	INT	✓	Chave estrangeira de usuário, responsável pela alteração
observacao	VARCHAR	✗	Observação registrada na alteração de status
alterado_em	TIMESTAMPTZ	✓	Data e hora da alteração, padrão data e hora atual

Fonte: Elaboração própria (2026).

4.4.5. Índices

Além das chaves primárias e das restrições de unicidade, foram criados índices nas colunas utilizadas com maior frequência nas consultas, conforme o Quadro 7.

Quadro 7 - Índices criados no banco de dados

Índice	Tabela	Coluna	Finalidade
idx_solicitacao_status	solicitacao	status	Filtro por status e indicadores do dashboard
idx_solicitacao_categoria	solicitacao	categoria_id	Filtro por categoria

idx_solicitacao_solicitante	solicitacao	solicitante_id	Visibilidade das solicitações por usuário
idx_solicitacao_criado_em	solicitacao	criado_em	Filtro por período e ordenação da listagem
idx_historico_solicitacao	historico_status	solicitante_id	Consulta do histórico de uma solicitação

Fonte: Elaboração própria (2026).

5. EXECUÇÃO E DEPLOY

5.1. Visão Geral

Esta seção descreve os procedimentos necessários para a instalação, a configuração e a execução do Portal de Solicitações Internas, em conformidade com os requisitos de deploy estabelecidos. São apresentados os pré-requisitos, a execução em ambiente local de desenvolvimento e o procedimento recomendado para implantação em servidor de produção.

A aplicação é composta por três componentes executados de forma independente: o banco de dados PostgreSQL, a API desenvolvida em Node.js e a interface desenvolvida em React. O código-fonte, os scripts de banco de dados e a documentação encontram-se reunidos no repositório do projeto, conforme o Quadro 8.

Quadro 8 - Informações gerais do projeto

Item	Detalhe
Repositório	https://github.com/majumelo/internal-portal-system
Ramificação principal	main
Versão do Node.js	20 ou superior
Versão do PostgreSQL	14 ou superior
Nome padrão do banco de dados	internal_bd

5.2. Pré-requisitos

Para a execução do sistema, é necessária a instalação prévia das ferramentas relacionadas no Quadro 8. As demais dependências são instaladas automaticamente pelo gerenciador de pacotes, a partir dos arquivos package.json do backend e do frontend.

5.3. Ambiente Local (Desenvolvimento)

O desenvolvimento e os testes foram realizados em ambiente local, com backend e frontend executados simultaneamente em terminais distintos.

5.3.1. Obtenção do código-fonte

```
git clone https://github.com/majumelo/internal-portal-system.git  
cd internal-portal-system
```

5.3.2. Criação do Banco de Dados

Deve ser criado um banco de dados vazio denominado internal_bd:

```
psql -U postgres -c "CREATE DATABASE internal_bd;"
```

Alternativamente, o banco pode ser criado pelo pgAdmin, por meio da opção Databases, Create, Database. As tabelas e os dados iniciais são criados na etapa seguinte.

5.3.3. Configuração do backend

```
cd backend  
npm install  
cp .env.example .env
```

Em sistemas Windows, a cópia do arquivo de configuração é realizada pelo comando Copy-Item .env.example .env, no PowerShell. Em seguida, devem ser ajustados no arquivo backend/.env o usuário, a senha e a porta do PostgreSQL na variável DATABASE_URL (postgres://postgres:sua_senha@localhost:5432/internal_bd).

5.3.4. Criação da estrutura do banco de dados

A estrutura do banco de dados e os dados iniciais são criados por um único comando:

```
npm run db:setup
```

Esse comando executa, em sequência, o script `database/01-schema.sql`, responsável pela criação das tabelas, restrições e índices; o script `database/02-seed.sql`, responsável pela inserção das categorias e dos usuários de demonstração; e o comando `prisma generate`, que gera o cliente de acesso ao banco. Os scripts podem ser executados mais de uma vez sem duplicação de dados.

Como alternativa, os scripts podem ser executados manualmente:

```
psql -U postgres -d internal_bd -f database/01-schema.sql
```

```
psql -U postgres -d internal_bd -f database/02-seed.sql
```

```
npx prisma generate
```

Os mesmos arquivos podem, ainda, ser executados na ferramenta Query Tool do pgAdmin, respeitando a ordem indicada.

5.3.5. Configuração do frontend

Em outro terminal, a partir da raiz do projeto:

```
cd frontend
```

```
npm install
```

```
cp .env.example .env
```

5.3.6. Execução

A execução do sistema requer dois terminais, conforme o Quadro 9.

Quadro 9 - Comandos de execução em ambiente localFonte: Elaboração Própria (2026).

Componente	Comando	Diretório	Endereço
Backend	npm run dev	backend/	http://localhost:3333

Frontend	npm run dev	frontend/	http://localhost:5173
----------	-------------	-----------	---

Fonte: Elaboração própria (2026).

A API é iniciada somente após confirmar a conexão com o banco de dados, exibindo no terminal a mensagem "API pronta na porta 3333; conexão com PostgreSQL confirmada.". Caso a conexão falhe, o processo é encerrado com a indicação do erro.

5.4. Variáveis de Ambiente

Os arquivos .env não são versionados no repositório. Os arquivos .env.example documentam todas as variáveis necessárias e devem ser utilizados como base para a configuração de cada ambiente.

Quadro 10 - Variáveis de ambiente do backend (backend/.env)

Variável	Descrição	Exemplo
PORT	Porta de execução da API	3333
DATABASE_URL	Endereço de conexão com o PostgreSQL	postgresql://postgres:sua_senha@localhost:5432/internal_bd
JWT_SECRET	Chave secreta de assinatura dos tokens	Texto longo e aleatório
JWT_EXPIRES_IN	Validade do token	8h
CORS_ORIGIN	Origens autorizadas, separadas por vírgula	http://localhost:5173

Fonte: Elaboração própria (2026).

Quadro 11 - Variáveis de ambiente do frontend (frontend/.env)

Variável	Descrição	Exemplo
VITE_API_URL	Endereço da API	http://localhost:3333

Fonte: Elaboração própria (2026).

Caso a senha do banco de dados contenha caracteres especiais, estes devem ser codificados na variável `DATABASE_URL`; o caractere "@", por exemplo, deve ser substituído por "%40". O valor de `VITE_API_URL` deve corresponder à porta definida em `PORT` no backend.

5.5. Portas Utilizadas

- Backend (Express) - 3333
- Frontend (servidor de desenvolvimento Vite) - 5173
- PostgreSQL - 5432

5.6. Acesso e Credenciais de Demonstração

Após a execução, o sistema pode ser acessado pelo endereço `http://localhost:5173`, utilizando os usuários de demonstração criados pelo *script* `02-seed.sql`, relacionados no Quadro 12.

Quadro 12 – Credenciais de demonstração

Usuário	Senha	Perfil
maria	123456	Colaborador
layanne	123456	Colaborador
ana	123456	Atendente

Fonte: Elaboração própria (2026).

Para a verificação do fluxo completo, recomenda-se o seguinte roteiro: autenticar-se como maria e registrar uma solicitação, editá-la ou excluí-la enquanto estiver com status "Aberto", encerrar a sessão e autenticar-se como ana, que visualiza todas as solicitações, alterando o status para "Em Atendimento" com uma observação; e, por fim, retorna como maria para constatar que a solicitação não pode mais ser editada e que o histórico registra a alteração realizada.

5.7. Problemas Comuns

O Quadro 13 relaciona os problemas mais frequentes durante a instalação e suas respectivas soluções.

Quadro 13 – Problemas comuns e soluções

Problema	Solução
Mensagem "Não foi possível conectar ao PostgreSQL"	Verificar se o PostgreSQL está em execução e se usuário, senha, porta e nome do banco em DATABASE_URL estão corretos
Erro EADDRINUSE	A porta já está em uso; encerrar o outro processo ou alterar PORT no backend e VITE_API_URL no frontend
Erro "@prisma/client did not initialize yet"	Executar npx prisma generate no diretório do backend
Erro de CORS no navegador	Incluir o endereço do frontend em CORS_ORIGIN e reiniciar o backend
Mensagem "Não foi possível conectar ao servidor" na autenticação	Confirmar que o backend está em execução e que VITE_API_URL aponta para ele; reiniciar o frontend após alterar o .env

Fonte: Elaboração própria (2026).